

Rapport du projet de SDD

Résumé du projet:

Le but du projet est de simuler un CPU et son fonctionnement de manière simplifiée.

Structure du projet:

Le projet est divisé en 4 catégories de fichier suivant:

- Hash : Table de Hachage, associe clé et valeur dans un tableau
- Handler : Gère l'espace alloué pour chaque segment de données
- Parser : Gère les données et le code
- CPU : L'élément principal du projet, va nécessiter tous les éléments précédents pour réaliser la gestion de données

Pour chaque catégorie, il y a un fichier de test associé que l'on exécute avec les commandes suivantes:

- make test_hash
- make test_handler
- make test_parser
- make test_cpu

Ou make all pour toutes les compiler.

Chaque test s'exécute sans fuite de mémoire.

Fonctions supplémentaires:

Voici toutes les fonctions supplémentaires et ce qu'elles font:

- int hashmap_size(HashMap *map) : Compte le nombre d'élément dans la table de hachage
- void memory_destroy(MemoryHandler *handler) : Libère la mémoire d'un memory handler
- int count_elements(const char *valeur) : Compte le nombre d'éléments dans une chaîne de caractère
- Segment *best_fit(MemoryHandler *handler, int size) : Trouve le meilleur segment libre pour allouer de la mémoire
- Segment *worst_fit(MemoryHandler *handler, int size) : Trouve le pire segment libre pour allouer de la mémoire

Explication série de test:

test_hash: on vérifie l'insertion d'élément, la récupération d'élément, suppression d'élément, la création et suppression du tableau.

test_handler: on vérifie l'initialisation et la suppression d'un handler, et de multiples création et suppression de segment.

test_parser: test count_élément (fonction personnelle), initialisation et suppression d'un parser, on crée un fichier .asm et on vérifie chaque fonction comme resolve_constants par exemple

test_cpu: initialisation et suppression d'un cpu, test des fonctions load et store, les différents types d'adressage, et exécution des séries de test contenu dans le sujet.